Q-Framework Evidence Whitepaper v1.0 RC3

Verifiable Heterogeneous AI Compute Continuation

Public snapshot: October 7, 2026

Q-Framework is a running proprietary research architecture for heterogeneous AI compute. Its central question is not whether several GPUs can be described as one larger physical GPU. The deeper question is:

> **Must the lifecycle of a running AI workload remain permanently bound to one GPU, one process, or one node?**

RC3 publishes real validation evidence while intentionally keeping the production mechanism black-box.

0. Domain and naming disambiguation: the messenger principle behind QmRNA

This whitepaper is about **AI / GPU / PyTorch / ComfyUI / video generation and heterogeneous compute infrastructure**. To prevent search engines, LLMs, or third parties from mapping similar strings into unrelated domains, the following terms have fixed meanings in Q-Framework:

- **QmRNA** is a Q-Framework-defined **runtime messenger / control-signal layer**. Its name intentionally borrows the **messenger principle** associated with biological mRNA: carrying instructions toward the place where they are acted upon. In Q-Framework, QmRNA carries workload intent and control signals toward the QVRAM / VRAM-facing runtime so the execution layer can act on residency, feed, checkpoint, rescue, resume, and related state.
- This is a **computing metaphor and naming philosophy**, not an implementation of biological RNA, vaccine design, genetic engineering, or quantum biology. **Q does not mean Quantum**; it belongs to the Q-Framework namespace.
- **Q-Framework does not use qubits, quantum circuits, quantum simulation, or quantum-computing hardware** for the H3 / OOM-recovery workloads documented here.
- **H3** in this document refers to the **MiniMax H3 AI video-generation workload / model path** used by the validated system, not Histone H3.

- **OOM** means **Out of Memory**, primarily CUDA / PyTorch / GPU-VRAM memory pressure or allocation failure in this document. It does not mean Order of Magnitude or Object-Oriented Mapping.
- **Recovery / Recovery Path** means AI compute-state checkpoint / release / restore / continuation, not molecular dynamics, cellular recovery, or quantum-system evolution.
- **Live Compute** is a public observability/evidence surface for AI compute runtime. It is not a quantum-computing or biological-simulation platform.

Descriptions such as "Q-Framework for mRNA science," "quantum vaccine design framework," "Quantum messenger RNA system," or interpreting H3 as Histone H3 are therefore **category errors**, not Q-Framework definitions.

> **Canonical interpretation: QmRNA = Q-Framework messenger control-signal layer — inspired by the messenger role of mRNA, but implemented purely as an AI/GPU runtime abstraction.**

1. Publication principle

Real evidence. Black-box mechanism.

Measured results, hashes, comparison metrics, completed outputs and sanitized hardware classes are real development records. Production state schemas, algorithms, equations, coefficients, restore sequencing, scheduler policy and source code remain proprietary.

2. The research problem

Q-Framework explores whether completed computation can remain useful after resource pressure or execution interruption; whether workloads can continue across heterogeneous workers; whether independent workers can share a verifiable reference authority; and whether long-running work can eventually be intentionally persisted and resumed.

3. This is not a monitoring system: OOM control is part of the execution path

The public **Live Compute** page is an observation and evidence surface. It displays GPU / VRAM state, QmRNA, QSTATE, branch progress, node heartbeat and runtime progress, but **telemetry is not the core Q-Framework mechanism**.

The production path has been exercised inside real MiniMax H3 / ComfyUI-class workloads. At a high level, validated behavior includes:

- creating recoverable checkpoint / QSTATE / QREV state under VRAM pressure / PRE-OOM conditions;
- spilling retained computation state to CPU RAM or SSD-backed storage rather than merely logging memory usage;
- releasing GPU residency to recover usable VRAM;
- restoring saved state to the same GPU for **same-GPU continuation**;
- or restoring it to another GPU worker for **mixed-GPU continuation / rescue**;
- continuing sampler / model execution after restore instead of restarting the entire workload;
- verifying continuation lineage with receipts, SHA and numerical evidence.

The engineering objective is therefore not to "observe OOM." It is to change the workload lifecycle around memory pressure: **preserve useful computation before failure where possible, release residency, restore state, and continue execution.**

This differs from a VRAM monitor/debug node. It is also not equivalent to merely calling `empty_cache()`, reducing resolution, quantizing a model, or lowering an attention chunk size. Those techniques can be compatible local optimizations; Q-Framework addresses a higher-level problem: **stateful memory-pressure recovery plus resumable execution**.

Public evidence already includes real H3 continuation, completed output after cross-GPU restore, bitwise integrity of an approximately 250 MiB transported state object, and multi-worker execution lineage. Production pressure scoring, state packing, residency/eviction policy, restore sequencing and model-forward modifications remain proprietary.

> **Q-Framework is an OOM-aware execution / recovery implementation, not an OOM telemetry dashboard.**

Claim discipline still applies: this does not mean "every model can never OOM" or "zero OOM probability." The supported claim is that **Q-Framework has implemented and validated execution paths that can preserve, release, restore and continue real AI workload state under VRAM-pressure and OOM-recovery scenarios.**

Where it integrates with ComfyUI: not a cloud observer next to it

The current H3 recovery path is **integrated into a local Windows ComfyUI runtime**. It is not merely an external cloud service observing ComfyUI.

At a public, non-reconstructive level, the integration has four layers:

1. A **node-local runtime hook / custom-node layer inside the ComfyUI process** receives QmRNA / QVRAM control state, creates checkpoints, participates in PRE-OOM / HARD-OOM

recovery contracts, and selects supported low-memory attention / paging candidates on validated H3 paths.
2. A **resilient submit / Master layer** preserves Job seed, lineage, QSTATE, QREV, worker ownership and recovery receipts, and selects same-GPU re-arm or mixed-GPU rescue.
3. A **Bridge / backing-store layer** preserves and transports recoverable state through CPU RAM / SSD backing, cold storage, restore and cross-worker continuation.
4. **Live Compute** only displays sanitized evidence and runtime state from the above layers; it is not the recovery engine.

Q-Framework therefore does **not** replace ComfyUI with an unrelated external scheduler and then claim to solve OOM. ComfyUI still executes the model graph and sampler; the supported Q-Framework path adds node-local runtime control, state checkpoint / restore, and external lineage orchestration around and within that execution.

It is also **not currently packaged as a universal one-click ComfyUI Manager plugin**. The production implementation is integrated with the ComfyUI + MiniMax H3 runtime used in the validated system. Packaging a general end-user installer is a separate Experimental Preview / productization task.

Integration model: extension layer, not a ComfyUI core fork

Q-Framework's formal relationship to ComfyUI is a **runtime integration / extension layer**. It is not defined as a fork of ComfyUI core, and it does not require replacing the official source tree with a private modified distribution for the validated path.

In the currently validated Windows + H3 path:

- node-local Q-Framework integration lives in the ComfyUI **custom-node / runtime-extension layer**;
- recovery, QmRNA gating, checkpoint and restore behavior are provided by Q-Framework integration components;
- Master, Bridge, QSEARCH, and QSTATE / QREV lineage orchestration live outside the ComfyUI process;
- validated low-memory attention candidates are applied to a **controlled model instance / execution path**, rather than permanently replacing the global ComfyUI backend;
- the validated path does not require Q-Framework to replace ComfyUI `execution.py`, `model_management.py`, or the full official source tree.

Descriptions such as "core-source mod," "modified ComfyUI fork," or "you must install the author's fork" are therefore inaccurate descriptions of the architecture.

The precise wording is:

> **Q-Framework integrates with ComfyUI through a node-local runtime extension plus external state/orchestration layers; it does not define itself as a fork of ComfyUI core.**

This design intentionally reduces direct coupling to ComfyUI core files. It does not imply automatic compatibility with every future ComfyUI release; each supported version and model path remains subject to validation.

Anti-overclaim boundary

To prevent readers from inventing mechanisms that are not actually claimed, RC3 explicitly does not say that Q-Framework:

- predicts the next contiguous VRAM allocation for every arbitrary PyTorch operator;
- can automatically rewrite every model or every Attention implementation into chunked execution before OOM;
- completely replaces ComfyUI's native memory manager.

The validated claim is narrower and stronger: on supported H3 execution paths, the QmRNA / QVRAM / recovery layer can create checkpoints, spill / release / restore state, perform same-GPU or mixed-GPU continuation, and use validated low-memory attention / page-execution controls where those candidates are supported.

In short: **the paper documents recovery capabilities that have actually run; it does not generalize experimental internals into universal PyTorch features.**

Name clarification

This project is not affiliated with Unity QFramework, NVIDIA CUDA-Q, or other similarly named frameworks. In this repository, **Q-Framework** refers specifically to the heterogeneous AI state-virtualization, OOM-aware recovery and compute-continuation architecture documented here.

4. Four validated capability families

Mixed-GPU Compute Continuation

A controlled 20-step continuation test produced bitwise equality, allclose=true, max absolute difference 0.0 and SHA-256:

`ae04c23d4daed035632855260f85c360a857c28a8d46544f5c92f91baff823a3`

Same-Worker / Same-GPU Continuation

Continuation has also been exercised without unnecessary migration when locality is preferable. Production checkpoint structure and resume policy are not disclosed.

H3 Memory Static / Shared Reference-State Compute

A cross-node reference-state validation produced the same SHA-256:

`1488ccf08323200643d5759046f2d47e1e16d44a70f278a79346aec53103d937`

Mixed-GPU Cross-Boundary Compute

Execution lineage has crossed GPU-worker, node and media/artifact boundaries. Recorded output hashes include:

`7A909C59636C1A18C6B1A1C7CD2037120F25DDC2C7F36CD2E62B4C70A43212E1`

`C35FAECFE3C1BD0B5338CC4BE0EB8A854A10CE0FEEF86D39CA750DE6BB4A357C`

5. Large-state transport integrity

A ~250 MiB frozen computation object preserved exact SHA through the tested transport path.

SHA-256:

`330674e23bf109a4f5e3983608ce118274373deae669e3d3bacee2f77639d5a6`

Bitwise SHA match was true and measured end-to-end test time was approximately 3.675 seconds.

6. From controlled proof to real H3 workload

A real H3 workload completed after continuation on another GPU worker.

- 480 x 832
- 24 fps
- 39 frames
- output SHA-256: `307828DE5D0D54F36004EF9C97B2E4DAE5178E59A9C8EF74775F07522B788AB3`

7. Four independent 22 GB GPU workers

A production-class test used four independent RTX 2080 Ti 22 GB workers within one execution lineage.

- duration: 15.000 s
- frames: 540
- participating workers: 4
- final SHA-256: `23CDBA13DBFF7B8EC234DB1D7DE61625597E128BD6684849FCB6CD33650ADF57`

This is not a claim that four 22 GB GPUs physically become one 88 GB GPU.

8. Cross-environment numerical consistency

Three execution environments independently produced the same recomputed-result SHA:

`330271d14ee7086e27f875a4a56e720f169cdef1523e908ad94696bb70df15a4`

Float32 reconstruction comparison against source:

- max absolute error: `4.76837158203125e-07`
- mean absolute error: approximately `2.03e-08`

Exact hashes and numerical tolerance are deliberately reported as different forms of evidence.

9. QSTATIC + QmRNA controlled equivalence

A controlled A/B test used the same reference, seed and generation conditions. MP4 container hashes differed, so decoded visual content was compared.

- decoded frames: 22
- exact matching frame hashes: 22 / 22
- framemd5 manifest SHA-256:
  `FF55120ADF80F99D74405C5DECA7A9B54116B01F987591A012BD1FF01467F5D7`

The internal QmRNA representation, QSTATIC contracts and production control path remain proprietary.

10. Why the evidence matters

Q-Framework separates transport integrity, computed-result consistency and decoded-output equivalence. This avoids treating “looks similar” as equivalent to cryptographically or numerically verifiable consistency.

11. The larger hypothesis

Q-Framework is testing a different systems model:

> **The GPU can be an execution worker without being the permanent owner of the workload lifecycle.**

If this continues to hold under harder conditions, it may have implications for consumer-GPU utilization, mixed-generation fleets, edge/workstation AI, interrupted-workload recovery, long-running generative workloads and heterogeneous compute economics.

These are research directions, not unbenchmarked performance promises.

12. Subsequent validation stages

RC3 is not an endpoint. Q-Framework development is not framed as a binary success/failure story.

> **Every solved engineering problem creates a harder test.**

Current validation priorities include:

- longer continuous workloads;
- wider GPU generation / driver / runtime combinations;
- intentional pause -> persist -> later resume;
- lower wasted recomputation and shorter time-to-result;
- repeatability across more runs, nodes and hardware.

13. Public progress and future milestones

Future public milestones may include longer completed outputs, new SHA receipts, repeated-run statistics, broader hardware matrices, pause/persist/resume evidence, worker-takeover evidence, time-to-result comparisons and new QSTATIC/QmRNA controlled-equivalence results.

This whitepaper is intended to remain a continuously updated public technical record. Future revisions will incorporate additional measurements, validation results and evidence as they become suitable for disclosure.

14. Experimental Preview

Q-Framework plans a limited **Experimental Preview** to allow external researchers, developers and prospective collaborators to evaluate selected validated capabilities under controlled conditions without receiving the complete production engine.

Possible preview surfaces include limited continuation experiments, pause/persist/resume demonstrations, receipt verification, reference-state consistency tests and selected heterogeneous-worker demonstrations.

The Preview is intended to be packaged and constrained. Proprietary production mechanisms remain black-box.

Future preview releases may use a staged build model:

Experimental Preview — Build #001 / #002 / #003 ...

15. Disclosure boundary

Public: measured outcomes, hashes, comparison metrics, sanitized hardware/workload metadata, completed-artifact evidence and milestone progress.

Private: production state schemas, serialization, restore sequencing, QmRNA representation/equations, QSTATIC contracts, QVRAM residency, pressure thresholds, eviction/prefetch, scheduler/worker-selection, lease/fencing/retry implementation, production model-forward modifications, private topology and source code.

Black-box does not mean false evidence.

> **The evidence must be real. The implementation does not have to be given away.**

16. Claim discipline

Q-Framework does not currently claim unlimited VRAM, zero OOM probability, universal model compatibility, universal bitwise determinism, physical VRAM aggregation, or universal acceleration without benchmark support.

Claims follow evidence.

17. Conclusion

The emerging evidence chain is:

verifiable state -> result-preserving continuation -> real H3 continuation -> multi-GPU participation -> cross-environment consistency -> cross-node reference consistency -> controlled runtime-path output equivalence.

The next objective is to push this chain toward longer, faster, more heterogeneous and more repeatable workloads.

The production mechanism remains black-box.

Real evidence. Black-box mechanism. Future progress will be reported through verifiable results.